MULTIMEDIA UNIVERSITY

CCP6224 Object Oriented Analysis and Design
Lab 5 – Key Listeners and Multimedia Libraries

Lab outcomes

By the end of today's lab, you should be able to

- Make use of Java Swing listeners – ChangeListener and KeyListener
- Make use of the Java MIDI library to generate musical notes that map to on screen components and keyboard interfaction

Keyboard listeners

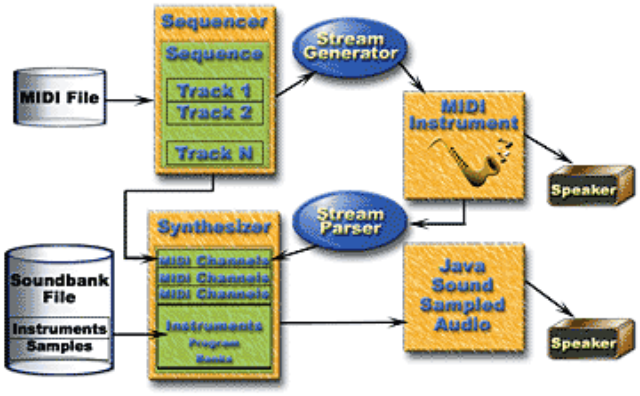
Keyboard activity is handled by the `KeyListener`. The listener is an interface object and needs to have all its methods overridden if they are "*implements*" in a concrete class. The methods in the `KeyListener` are

```
void keyTyped (KeyEvent e)
void keyPressed (KeyEvent e)
void keyReleased (KeyEvent e)
```

Keys that are pressed can be filtered and detected using key patterns (VK patterns) or through character matching (i.e. extract the character that was pressed and then perform matching)

Multimedia libraries

The Java Sound programming interface supports play and record Wave (WAV), Audio Utility (AU), Apple Interchange File Format (AIFF), and other sampled audio formats. Additionally, Java Sound supports the Music Instrument Digital Interface (MIDI) standards to represent music not as a digital audio file but as a transcription of messages representing musical qualities such as notes, durations, velocity, speed etc.



To implement MIDI playback in any Java program, first a `Synthesizer` object is created. Each synthesizer plays/records MIDI data on a single channel (upto 16 are available … actually 15) using one of 127 available instruments. Additional channels/instruments are available through extensions. More details are available at https://docs.oracle.com/javase/10/docs/api/javax/sound/midi/MidiChannel.html

Exercise

The following code presents the starter framework for a Java Swing application to create a MIDI piano. Save the code into a file, compile and execute and you should get the GUI shown below

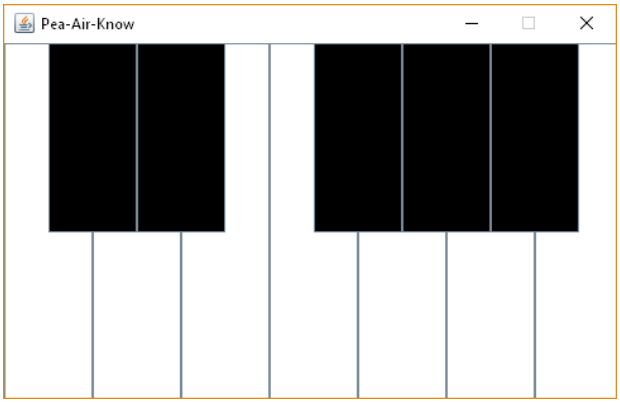
```
import javax.swing.*;
import javax.swing.event.*;
import java.awt.event.*;
import java.awt.*;

public class Piano {
	Piano(){
		JFrame frame = new JFrame("Pea-Air-Know");
		JButton[] w = new JButton[7];
		JButton[] b = new JButton[6];
		JLayeredPane panel = new JLayeredPane();
		frame.add(panel);

		for (int i = 0; i < 7; i++) {
			w[i] = new JButton();
			w[i].setBackground(Color.WHITE);
			w[i].setLocation(i * 70, 0);
			w[i].setSize(70, 300);
			panel.add(w[i], 0, -1);
		}

		for (int i = 0; i < 6; i++) {
			if (i==2)
				continue;
			b[i] = new JButton();
			b[i].setBackground(Color.BLACK);
			b[i].setLocation(35 + i * 70, 0);
			b[i].setSize(70, 150);
			panel.add(b[i], 1, -1);
		}

		frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
		frame.setSize(500, 320);
		frame.setResizable(false);
		frame.setVisible(true);
	}
	public static void main(String[] args) {
		new Piano();
	}
}
```



The following items need to be added to complete the program to create a working piano in Java

- Add a `ChangeListener` to the code and attach the listeners to all the buttons instantiated (we cannot use `ActionListener` here because the `actionPerformed` method does not indicate when a component is pressed/released)
- Override the `stateChanged` method from `ChangeListener` to handle the `JButton` presses (use `System.out` to debug your `stateChanged` method for now)
- "Name" the individual `JButtons` using the `setName()` method to allow identification later in the `stateChanged` method
- Import the MIDI library into the program and initialize a MIDI synthesizer object:
  - `import javax.sound.midi.*;`
  - `Synthesizer synth;`
  - `MidiChannel[] mChannels`
  - ```
    try
    { synth=MidiSystem.getSynthesizer();
      synth.open();
      mChannels = synth.getChannels();
    }
    catch (MidiUnavailableException e)
    { JOptionPane.showMessageDialog(null,"Unable to open MIDI.");
    }
    ```
- Map the MIDI method `noteOn` and `noteOff` to the `stateChanged` methods to play a corresponding note when a button is pressed.
  - `synth.getChannels()[0].noteOn(65,127);` // 65 is a note number, 127 is a loudness
  - `synth.getChannels()[0].noteOff(65);` // 65 is a note number
  - The first button on the left is 60 and the last button is 71.
- Implement the `KeyListener` and override the methods `keyPressed`, `keyTyped` and `keyReleased`. You will need to `setFocusable(true);` on the JFrame for it to "hear" the keypresses.
- Map the code from `stateChanged` to the `keyPressed` and `keyReleased` methods
    ```